Project Documentation

1. Overview

Rural Mental Health Screening AI is a browser-based and backend-supported screening platform built for rural and resource-co

It combines:

- multilingual questionnaire intake,
- narrative text analysis,
- optional audio and image capture,
- passive biomarker support,
- adaptive screening,
- model statistics and quality checks,
- PDF reporting,
- record lookup and export,
- offline-first fallback support,
- a research-ready paper and evaluation toolkit.

The project is designed so a health worker can complete a screening session in one flow, review the result immediately, save i

2. Project Goals

The system is built around these goals:

1. Support rural screening workflows with low friction.
2. Present the workflow in English, Hindi, and Bengali.
3. Combine questionnaire scoring with text, audio, image, and passive signals.
4. Offer live preview before saving.
5. Keep the dashboard usable when backend connectivity is weak or unavailable.
6. Persist assessment records for later retrieval.
7. Provide PDF exports for records and quality checks.
8. Support research publication, ablation studies, and future clinical validation.

3. Main Components

3.1 Frontend Dashboard

Files:

- web/index.html (/c:/Rural%20Mental%20Heath%20Screening%20AI/web/index.html)
- web/app.js (/c:/Rural%20Mental%20Heath%20Screening%20AI/web/app.js)
- web/styles.css (/c:/Rural%20Mental%20Heath%20Screening%20AI/web/styles.css)

Responsibilities:

- render the app shell and dashboard layout,
- switch between languages,
- collect assessment inputs,
- render live previews,
- render analytics and report panels,
- hide demo records from the visible user list,
- hide report-only entries from the Records and Reports panel,
- allow record deletion and report removal.

3.2 Backend Server

File:

- dashboard_server.py (/c:/Rural%20Mental%20Heath%20Screening%20AI/dashboard_server.py)

Responsibilities:

- serve the dashboard frontend,
- validate assessment payloads,
- compute preview and saved-screening outputs,
- call storage and report utilities,
- expose API routes,
- generate PDFs,
- provide model-stats and quality-check endpoints,
- support adaptive screening requests,
- serve health and manifest endpoints.

3.3 Core Python Package

Folder:

- src/mental_health_screening/ (/c:/Rural%20Mental%20Heath%20Screening%20AI/src/mental_health_screening)

Important modules:

- assessment.py: questionnaire structure and scoring helpers.
- feature_extract.py: text/audio/image feature extraction and preprocessing.
- predict.py: heuristics, confidence estimation, and risk aggregation.
- storage.py: SQLite and persistence helpers.
- report.py: PDF report generation.
- training.py: model-bundle training utilities.
- model_store.py: bundle metadata, manifest summaries, and model-stat helpers.

- utils.py: shared utilities used across the package.

3.4 Supporting Directories

- data/ (/c:/Rural%20Mental%20Heath%20Screening%20AI/data)
- models/ (/c:/Rural%20Mental%20Heath%20Screening%20AI/models)
- tools/ (/c:/Rural%20Mental%20Heath%20Screening%20AI/tools)
- tests/ (/c:/Rural%20Mental%20Heath%20Screening%20AI/tests)
- examples/ (/c:/Rural%20Mental%20Heath%20Screening%20AI/examples)
- paper/ (/c:/Rural%20Mental%20Heath%20Screening%20AI/paper)

These contain manifests, sample results, trained bundle outputs, evaluation scripts, training helpers, test files, and the researc

4. Working Functionality

This section explains how the system behaves in practice.

4.1 Assessment Creation

The Assessment Workspace is the main entry point.

What the user does:

1. Select a preferred language.
2. Enter profile details such as name, age, village, assessor, and consent.
3. Answer the questionnaire items.
4. Add a written narrative.
5. Optionally provide audio, face image, or passive biomarker inputs.
6. View the live preview.
7. Save the assessment.

What the system does:

- validates required fields,
- scores the questionnaire,
- extracts or estimates text signals,
- computes modality confidence and quality,
- merges the signals into a multimodal result,
- generates recommendation text and disclaimer,
- saves the record,
- updates analytics and records panels.

4.2 Live Preview

Before saving, the dashboard can generate a preview from the current browser inputs.

This preview is used to:

- show the likely screening outcome,
- display NLP and safety insights,
- show modality readiness,
- help the user identify missing inputs,
- keep the user aware of whether the backend is available.

If the backend is unavailable, the app falls back to offline heuristics and clearly labels the result.

4.3 Adaptive Test

The adaptive test asks one question at a time.

Working behavior:

- it loads the first question,
- collects the answer,
- chooses the next question based on the previous response state,
- continues until the adaptive session is complete,
- then saves the final record.

This reduces the number of unnecessary questions and makes the screening flow more field-friendly.

4.4 Analytics Hub

The analytics hub is the detail view for one selected assessment.

It shows:

- the final combined result,
- the questionnaire score breakdown,
- component contribution from text, audio, image, and passive signals,
- modality quality and availability,
- NLP and safety signals,
- recommendation and disclaimer,
- model statistics,
- quality check summary,
- trajectory and trend summary,
- feature snapshots and record details.

The analytics page is intentionally more verbose than the assessment workspace because it is meant for review and analysis.

4.5 Records and Reports

The records panel is used to:

- search and fetch records by assessment ID,
- inspect selected assessment details,
- export PDF reports,
- delete a record,
- hide a report from the visible panel.

Important visibility rules:

- demo/backend records are hidden from the normal user list,
- hidden report entries are removed from the visible Records and Reports panel,
- only visible user records are used in the main record list,
- demo records are excluded from PDF export.

5. Data Flow

The system uses this overall flow:

1. User starts a session in the browser.
2. Frontend builds a payload from profile, questionnaire, narrative, and optional media inputs.
3. Payload is sent to /api/preview or /api/assessments.
4. Backend validates the input.
5. Feature extraction and scoring produce modality and overall outputs.
6. The record is saved in storage.
7. The frontend refreshes the result list and detail panels.
8. The user can fetch, review, export, delete, or hide the record later.

If the backend cannot be reached:

- the dashboard can still render a limited preview,
- the assessment can be stored locally,
- the app queues it for later sync,
- the UI clearly marks the offline state.

6. API Reference

The backend exposes these routes:

- GET / - dashboard shell
- GET /health - health check

- GET /sw.js - service worker file
- GET /manifest.webmanifest - app manifest
- GET /api/assessments - list saved assessments
- POST /api/assessments - create and save an assessment
- POST /api/preview - preview without saving
- GET /api/adaptive/config - adaptive screening configuration
- POST /api/adaptive/next - adaptive next-question selection
- GET /api/validated-instruments - validated instrument list
- GET /api/assessments/<assessment_id> - fetch a saved record
- DELETE /api/assessments/<assessment_id> - delete a saved record
- GET /api/assessments/<assessment_id>/trajectory - longitudinal trajectory
- GET /api/assessments/<assessment_id>/report.pdf - PDF report
- GET /api/database - database metadata
- GET /api/database/audit-logs - audit log summary
- GET /api/database/backups - backup listing
- POST /api/database/backup - create a backup
- GET /api/model-stats - model bundle statistics
- GET /api/quality-check - quality-check summary
- GET /api/quality-check/report.pdf - quality-check PDF export
- GET /api/sample - bundled sample records

7. Frontend Behavior in Detail

7.1 Language Switching

The UI supports:

- English,
- Hindi,
- Bengali.

Language affects:

- navigation labels,
- questionnaire display,
- validated instrument names,
- recommendations,
- disclaimers,
- quality-check text,
- offline messages,
- record visibility labels,
- adaptive-screening prompts.

7.2 Validated Instruments

The validated instrument panel supports localized display of PHQ-9 and PHQ-4.

The UI is designed so that if the system language changes, the instrument title and explanation are shown in the matching lang

7.3 Record Cards and Visibility

The frontend now distinguishes between:

- visible user records,
- demo records,
- report-hidden records.

This ensures the user only sees real user records in the main records/report flow.

7.4 Recommendation Panel

The recommendation panel displays:

- a follow-up recommendation,
- a screening disclaimer,
- a source summary showing which signals contributed,
- a narrative-availability note.

The text is risk-aware:

- high risk asks for urgent follow-up,
- moderate risk recommends prompt follow-up and repeat screening,
- low risk recommends ongoing support and rescreening only if symptoms persist or worsen.

8. Scoring and Signal Processing

The system blends several sources of evidence:

8.1 Questionnaire

The questionnaire is scored into per-domain scores and per-domain risk levels.

8.2 Text NLP

The narrative text pipeline can include:

- sentiment,
- dominant emotion,

- emotion intensity,
- safety language,
- self-harm keywords,
- distress phrases,
- agrarian distress signals,
- transformer-backed features when available.

Runtime priority:

- text_transformer is the primary scorer when installed locally
- text remains the fallback bundle
- the weakest text domains are lightly retuned from existing text-side cues instead of changing the full text bundle

8.3 Audio

Audio inputs can contribute:

- file metadata,
- duration,
- voiced ratio,
- quality-related values,
- backend inference when available.

Runtime priority:

- audio is the primary scorer
- audio_spectrogram and audio_sequence remain secondary support
- mood-heavy audio domains receive the most benefit from the main audio bundle

8.4 Image

Image inputs can contribute:

- file metadata,
- vision backend markers,
- face-related cues,
- image quality indicators,
- backend inference when available.

Runtime priority:

- image_dl is the primary scorer
- the classical image bundle remains a fallback and stabilizer

8.5 Passive Biomarkers

Passive inputs can include:

- typing rhythm,
- passive video metadata,
- rPPG-style or movement-style indicators when available.

8.6 Fusion

The system combines these sources into:

- per-domain scores,
- overall confidence,
- final recommendation,
- disclaimer,
- modality quality cards,
- trend/trajectory views.

The fusion layer currently:

- keeps text close behind the stronger visual and acoustic signals,
- gives audio a slightly larger voice in the final blend,
- gives image DL slightly more influence than the classical image bundle,
- keeps comorbidity as a downstream summary of both the chain ensemble and upstream modality consensus.

9. Offline-First Behavior

The app is designed to remain usable even if the backend is temporarily unavailable.

When offline:

- the dashboard can show an offline preview,
- assessments can be queued locally,
- the UI shows that the assessment is pending sync,
- the app can recover when the connection returns.

This is important for rural deployments and mobile field use.

10. Storage and Record Management

The backend persists records using the projects storage layer.

Key capabilities:

- create a new assessment record,
- fetch a record by assessment ID,
- list saved assessments,
- build trajectory summaries,
- delete a record,
- create backups,
- audit access where supported.

The UI also maintains its own hidden-report state in browser storage so a report can be removed from the visible panel without

11. Model and Bundle Metadata

For a dedicated explanation of the deep learning model families themselves, see docs/DEEP_LEARNING_MODEL_GUIDE.md

The project keeps track of:

- trained bundle metadata,
- manifest paths,
- dataset roots,
- sample counts,
- modality coverage,
- confidence hints,
- model family labels,
- macro R2 and other bundle statistics.

These details appear in the model-statistics section so the user can understand how much of the result comes from trained bun

The same trained sklearn estimators also have ONNX exports in models/mental_health_screening/onnx/, and the original .pkl

See also:

- docs/MULTIMODAL_FUSION.md (MULTIMODAL_FUSION.md) for the current bundle priorities and fusion rules.

12. Quality Check Functionality

The dashboard includes a quality-check workflow.

It can:

- evaluate saved assessments,
- compute proxy metrics,
- show accuracy,
- show macro F1,

- show Brier score,
- show ROC AUC,
- list mismatched examples,
- export a quality-check PDF.

This is useful for validation, monitoring, and research reporting.

13. Research and Paper Support

The repository also includes a research-paper starter package:

- paper/manuscript.tex (/c:/Rural%20Mental%20Heath%20Screening%20AI/paper/manuscript.tex)
- paper/references.bib (/c:/Rural%20Mental%20Heath%20Screening%20AI/paper/references.bib)
- paper/research-proposals.md (/c:/Rural%20Mental%20Heath%20Screening%20AI/paper/research-proposals.md)
- paper/ethics-and-data-availability.md (/c:/Rural%20Mental%20Heath%20Screening%20AI/paper/ethics-and-data-availability.m
- paper/submission-checklist.md (/c:/Rural%20Mental%20Heath%20Screening%20AI/paper/submission-checklist.md)

That package is meant to support:

- manuscript drafting,
- literature review,
- future-work proposals,
- ethics and data-availability statements,
- publication checklisting.

14. Training and Evaluation Utilities

The repository includes utilities for:

- training,
- manifest generation,
- federated or multi-center experiments,
- saved-assessment evaluation,
- quality check generation.

Useful scripts include:

- tools/run_real_training_pipeline.py (/c:/Rural%20Mental%20Heath%20Screening%20AI/tools/run_real_training_pipeline.py)
- tools/run_federated_training.py (/c:/Rural%20Mental%20Heath%20Screening%20AI/tools/run_federated_training.py)
- tools/evaluate_saved_assessments.py (/c:/Rural%20Mental%20Heath%20Screening%20AI/tools/evaluate_saved_assessme
- tools/run_quality_check.py (/c:/Rural%20Mental%20Heath%20Screening%20AI/tools/run_quality_check.py)

15. Sample and Demo Data

The repository contains sample and demo assets for development and testing.

Important note:

- demo records are intentionally hidden from the user-facing record list,
- demo records are not included in report export,
- demo data should not be treated as real clinical data in any research output.

16. Deployment and Setup

Typical setup:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python dashboard_server.py
```

Then open:

http://127.0.0.1:8000/

17. Operational Notes

Before using this project in a real setting:

- verify consent workflows,
- confirm privacy and data-handling policy,
- test offline recovery,
- confirm PDF export behavior,
- test the hidden-report and delete-record actions,
- check model bundle availability,
- make sure sample data is not mistaken for real patient data.

18. Current Working Behavior Summary

At runtime, the system now behaves as follows:

- text and image are treated as usable when they carry actual signal or metadata,
- recommendations adapt to risk level and language,
- model statistics and NLP summaries avoid empty placeholder states when data is available,
- demo records stay hidden from the user-facing list,
- reports can be hidden from the visible panel without deleting the underlying record,
- the dashboard can fall back to offline preview when necessary,
- the research paper package and documentation are included in-repo.

19. Suggested Additions If You Want Even More Detail

If you want the documentation to become a full technical manual, the next files to add would be:

- docs/ARCHITECTURE_DIAGRAM.md
- docs/API_REFERENCE.md
- docs/DATA_SCHEMA.md
- docs/MODEL_PIPELINE.md
- docs/DEPLOYMENT_GUIDE.md
- docs/RESEARCH_WORKFLOW.md